

Deadline-Aware Token Scheduling for Physical-I/O LLM Agents

Md Ashikul Haque and Abusayeed Saifullah
Department of Computer Science
University of Texas at Dallas, Richardson, TX, USA

Abstract—*Large language model (LLM) agents increasingly issue actions for robots, mobile interfaces, and sensing systems. In these physical-I/O loops, the deadline applies to the first useful action, not to the final token of a response. Existing LLM-serving systems improve batching, key-value (KV) cache use, throughput, and tail latency, but they do not admit work or reserve capacity for the prefix that makes an action usable. We present Deadline-Aware Token Scheduling (DATS), a scheduling layer that decomposes each request into prefill, a validator-backed mandatory prefix, and an optional suffix. DATS admits a critical request only when its prefill, mandatory tokens, and validation cost satisfy a deadline guard and its KV-cache reservation fits in memory. Admitted mandatory work is dispatched in EDF order, and optional tokens run only when slack remains. DATS-MF extends DATS with three certified service modes (full, compact, and safe fallback) so a critical request can degrade gracefully when its full mode is infeasible. The analysis gives a sufficient schedulability condition for the EDF-compatible mandatory mode under bounded service and blocking. Across deterministic workloads, single-mode DATS has zero admitted-critical mandatory misses, and rejected critical requests count toward residual failure. DATS-MF reduces this critical failure rate from 0.402 for full-only DATS and from 0.399 for guarded segmentation without fallback to 0.119, by converting some critical rejections into compact or fallback service that still meets the deadline. On a vLLM/ROCm stress test of 5,760 requests and 13,626 token events, DATS-MF reaches 0.238 critical failure, compared with 0.925 for full-only DATS, guarded segmentation without fallback, and FCFS. Compact and certified fallback modes convert critical rejections into degraded but timely service. DATS reports admission, memory reservation, and mode selection through one scheduling layer and reports critical failure over all critical arrivals.*

Index Terms—Real-time scheduling, large language model serving, mandatory/optional computation, physical-I/O agents, admission control, deadline-aware scheduling, mixed criticality.

I. INTRODUCTION

Large language model (LLM) agents are moving from chat interfaces to systems that touch the physical world. A warehouse robot may need to query an LLM for a stop or maneuver command before the next control tick at 100 to 250 ms, set by the wheel-actuation loop. A smartphone screen-reader assistant may need to confirm a one-step UI action before the screen flips at the next animation frame, typically within 500 ms to 2 s. An industrial IoT scheduler may need to pick the next sensing channel within tens of milliseconds, before a frequency-hopping radio moves on. In each case the deadline is not chosen by the LLM serving stack: it is the period of an external physical process (actuation loop,

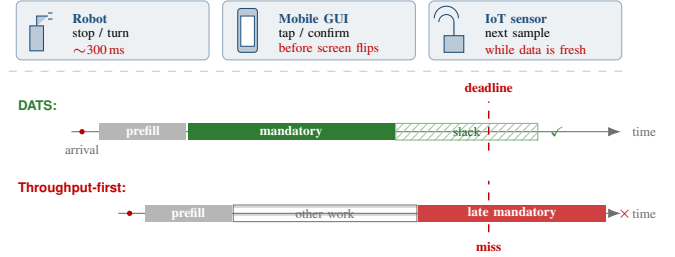


Fig. 1. Top: three example physical-I/O agents and their action deadlines. Bottom: DATS (green) meets the deadline; throughput-first (red) runs other work first and misses.

frame refresh, radio dwell), and the useful output is a short action prefix. Producing a polished response after that external deadline does not help the physical loop.

Fig. 1 illustrates three example physical-I/O agents and compares two scheduling strategies on the same request. A throughput-first server can keep generating tokens after the physical deadline has passed, because the mandatory prefix arrives too late. A real-time scheduler can instead stop after the mandatory prefix and use any remaining slack for explanation.

Existing LLM serving systems make generation efficient. Orca [1] and vLLM [2] provide iteration-level scheduling, batching, and key-value (KV) cache management. These mechanisms improve throughput and memory use, but they do not reserve service for the part of a response that makes a physical action possible. Recent time-sensitive LLM-serving work, including TimelyLLM, shows that robotic and physical-I/O agents need scheduling decisions during generation [3], [4]. No scheduling layer admits work or reserves capacity for the prefix that makes an action usable.

Classical real-time scheduling provides deadlines, demand, laxity, and blocking models [5]–[7]. Imprecise-computation scheduling decomposes jobs into mandatory and optional parts [8]. Real-time machine learning systems also model criticality, accelerator sharing, and memory pressure [9]–[12]. An LLM request combines these issues with prompt-dependent prefill, token-by-token decoding, unknown decoding length, growing KV state, and possible tool or I/O blocking. EDF or LLF applied to the whole response does not separate the useful prefix from the rest.

Many physical-I/O responses have a minimum useful prefix and an optional suffix. A robot may need a stop or maneuver command before the next control tick. A mobile agent may need a one-step UI decision. A sensor scheduler may need

the next sensor choice, not the explanation. Tokens before the minimum useful point are mandatory for the timing objective. Tokens after that point can improve explanation, confidence, or user-facing quality only if slack remains. Treating all tokens as mandatory wastes capacity during overload; treating the entire response as best effort fails to protect critical prefixes.

We present Deadline-Aware Token Scheduling (DATS), a scheduling layer for LLM agents with physical deadlines. DATS decides whether to admit a request, run prefill, emit a mandatory token, spend slack on optional tokens, or reject a request that would violate the guard. DATS reserves conservative prefill and minimum-token demand for protected critical work and checks memory feasibility at admission. During dispatch, an EDF-compatible protected mode serves mandatory prefixes first, and optional tokens are scheduled only when the mandatory guard remains feasible.

For overload, we extend DATS with *Multi-mode Fallback* (DATS-MF), which certifies three service modes (full, compact, and safe fallback). A critical request may carry several certified modes: a full LLM prefix, a compact schema prefix, or a safe fallback action. DATS-MF treats each candidate mode as a separate admission target and admits the highest-utility mode whose mandatory prefix is feasible. When even the lowest-utility certified mode would violate the guard, the request is rejected before optional text consumes service reserved for imminent deadlines.

Critical failure in DATS includes both admitted critical misses and rejected critical requests, so the metric reflects what the physical loop actually sees. Utility before the minimum useful prefix is zero, and optional tokens after the deadline do not count toward optional completion. A late or incomplete action is not credited as a successful control decision.

We implemented DATS in a deterministic simulator and used request and event records to evaluate admission, dispatch, and overload behavior. The multi-mode DATS-MF evaluator adds mode selection, critical fallback, reservation, and memory-guard ablations. We also exercise the runtime interface through local CPU calibration, llama.cpp [13] and Vulkan [14] telemetry, client-side chunked generation control, an in-process engine loop, and vLLM [2], [15]/ROCm [16] scheduler-hook integration.

In summary, the paper makes the following contributions:

- 1) We formulate a validator-backed mandatory-prefix model for physical-I/O LLM requests. The model makes prefill, decode, optional utility, criticality, and per-mode KV-cache reservation explicit at the scheduling interface.
- 2) We design DATS, a scheduling layer that admits a protected mandatory prefix only when it passes a joint demand and memory guard, dispatches the EDF-compatible protected mode first, and spends slack on optional service only while the guard remains feasible.
- 3) We extend DATS to DATS-MF, which treats full, compact, and certified safe-fallback modes as separate admission candidates so a critical request can degrade gracefully before rejection.

- 4) We prove a sufficient demand-bound condition for the EDF-compatible mandatory mode and state the blocking, memory-reservation, and critical-isolation conditions required for the guarantee.
- 5) We evaluate DATS and DATS-MF on deterministic workloads, ablations, validator-derived prefixes, runtime telemetry, an in-process engine loop, and a vLLM/ROCm scheduler-hook deployment against EDF, LLF, SPOF, batching, segmented TimelyLLM-style, guarded, and fallback baselines.

The rest of the paper is organized as follows. Section II reviews background and related work. Section III formalizes the request, timing, utility, and memory model. Section IV details DATS’s admission and dispatch. Section V proves the schedulability condition. Section VI covers the implementation. Section VII reports the evaluation. Section VIII concludes the paper.

II. BACKGROUND AND RELATED WORK

LLM serving systems. Orca introduced iteration-level scheduling and selective batching for transformer generation, and vLLM/PagedAttention showed that paged KV-cache management can substantially improve LLM serving memory efficiency and throughput [1], [2]. These mechanisms motivate the runtime interface used by DATS: the scheduler assumes generation can be observed at prefill/decode boundaries and that KV-cache occupancy is a first-class resource. Orca and vLLM target serving efficiency, not admission or schedulability for physical-I/O deadlines.

Real-time language-model inference. RT-LM studies uncertainty-aware resource management for real-time inference of language models [17]. TimelyLLM targets time-sensitive LLM serving for physical-I/O agents through segmented generation and scheduling [3], [4]. DATS focuses on mandatory/optional token accounting, admission and memory feasibility, deadline metrics that include rejected critical requests, and request-stream weakly-hard behavior. TokenButler motivates token utility and KV-cache sparsity as related mechanisms [18].

Classical, imprecise, and modern real-time scheduling. EDF, laxity-based dispatch, and demand-bound arguments provide the baseline priority vocabulary once LLM work is decomposed into bounded prefill and decode quanta [5]–[7]. Imprecise computation [8] provides the mandatory/optional abstraction. Execution effects that appear in token-serving systems are covered by work on non-preemptive blocking [19], batching [20], prediction [21], weakly-hard behavior [22], and probabilistic deadline failure [23], [24]. DATS reports the LLM-specific units, minimum useful output, optional budget, and memory growth needed before those real-time rules can be applied. It uses EDF-compatible mandatory ordering for analyzable claims and weighted optional scoring only as a slack policy.

Real-time ML and accelerator resource management. Prior work on real-time ML studies criticality-aware inference,

TABLE I
CLOSEST-PRIOR POSITIONING. DATS DIFFERS BY REPORTING
MANDATORY PREFIXES, OPTIONAL SUFFIXES, ADMISSION, REJECTION,
AND KV-CACHE MEMORY TOGETHER AT A SINGLE SCHEDULING LAYER.

Prior line	What it contributes	Gap addressed by DATS
Orca/vLLM [1], [2]	Iteration-level serving, batching, KV-cache efficiency.	No admission/schedulability model for physical deadlines or minimum useful prefixes.
RT-LM [17]	Real-time resource management for LM inference under uncertainty.	Does not model physical I/O mandatory/optional token prefixes and KV-aware admission at decode granularity.
TimelyLLM [3], [4]	Time-sensitive LLM serving for physical-I/O/robotic settings.	DATS focuses on mandatory/optional token accounting, KV-aware admission, critical failure over all critical arrivals, and request-stream weakly-hard evaluation.
Real-time ML pipelines [9]–[12]	Criticality, DNN scheduling, accelerator memory/resource control.	Workloads are DNN/perception pipelines rather than autoregressive prefill/decode streams.
Imprecise and real-time scheduling [5], [8], [19]–[21]	Mandatory/optional work, deadline ordering, blocking, batching, prediction-aware scheduling.	Requires exposing LLM-specific units, utility, and memory before rules are safely applicable.
ROS/physical systems [32], [33], [37], [38]	End-to-end timing models for robot/executor workloads.	Does not schedule token-generating LLM service inside the agent pipeline.

DNN execution, and accelerator memory management. Algorithmic priority inversion shows that ML pipelines can invert application criticality if model-internal work is not scheduled explicitly [9]. Self-cueing and canvas-based attention scheduling treat perception as a selective computation problem under timing constraints [25], [26]. LaLaRAND [10], DNN-SAM [27], RT-MOT [28], demand layering [11], GPU partitioning [29], GPU memory oversubscription [30], RT-Swap [12], and HARD [31] address accelerator and memory behavior for real-time ML workloads. DATS uses similar resource accounting but targets autoregressive token streams rather than DNN frame pipelines.

Physical-I/O and robotics runtimes. ROS 2 and autonomous-system scheduling papers define callback chains, resource interference, and end-to-end latency semantics for application-facing timing models [32]–[38]. DATS focuses on the LLM serving layer that such agents call. It makes request deadlines, prefill/decode costs, and KV-cache demand explicit so a downstream executor or control loop can reason about whether useful output can arrive in time.

Overload and mixed criticality. Mixed-criticality scheduling, CPS resource allocation, and deadline-miss handling work motivates explicit overload accounting rather than silent queue growth [39]–[41]. DATS reports critical failure over all critical arrivals, including rejected critical requests, precisely to avoid hiding overload behind a zero admitted-miss metric. Table I positions DATS against the closest prior lines of work.

TABLE II
REQUEST METADATA USED BY DATS.

Symbol	Meaning
r_i, d_i	Arrival time and absolute physical deadline.
$\mathcal{M}_i$	Candidate service-mode set for request i (full, compact, fallback).
P_i	Prefill service demand before the first token.
$c_i(k)$	Service demand for decode token k .
$K_i^{\min}, K_i^{\max}$	Minimum useful token threshold and maximum token budget.
$U_i(k)$	Utility after k emitted tokens.
$M_{i,m}(k)$	Prompt plus generated-token KV/runtime footprint after k tokens in mode m .
$V_{i,m}^+$	Bounded validator and fallback action-dispatch cost.
L_i	Criticality or service class.

III. SYSTEM MODEL

We consider an LLM serving stack that sits between a physical-I/O agent (a robot controller, mobile UI agent, or IoT scheduler) and the generation engine. A *request* is one LLM call by such an agent: a prompt and context for which the engine must produce output tokens, and on which the agent acts. A *deadline* is the latest moment at which the emitted prefix can still affect the external physical process the agent serves, for example the next actuation tick of a control loop or the next radio dwell of a sensor; the scheduler treats it as input. The rest of this section makes these notions formal.

Each request i arrives at time r_i and has absolute deadline d_i . It carries a candidate mode set $\mathcal{M}_i$ with elements m covering full generation, compact schema generation, or a safe fallback action. In single-mode DATS, $|\mathcal{M}_i| = 1$. Mode m requires prefill service $P_{i,m}$, then emits decode tokens with costs $c_{i,m}(k)$. It reaches minimum usefulness after $K_{i,m}^{\min}$ tokens and may continue until $K_{i,m}^{\max}$. The single-mode notation $P_i, c_i(k), K_i^{\min}$, and $K_i^{\max}$ is used when the mode is clear. Utility after k tokens is $U_{i,m}(k)$. Memory $M_{i,m}(k)$ denotes the total prompt plus generated-token KV footprint after prefill and k emitted tokens. Table II summarizes the metadata visible to the scheduler.

A. Request Lifecycle

A request has three phases within the selected mode. First, the serving engine performs prefill on the prompt and context. Second, the request enters mandatory decode until $K_{i,m}^{\min}$ tokens have been emitted. Third, if time and resources remain, the request may receive optional decode until $K_{i,m}^{\max}$ or until its utility saturates. A request is mandatory-successful only if prefill, the selected mode’s minimum useful prefix, and bounded validation or fallback-dispatch work finish by d_i , and a validator accepts the emitted prefix. A free-form prefix from a critical request is not counted as successful unless a validator accepts it. Completing optional tokens after the minimum prefix can improve utility but cannot convert a missed mandatory deadline into a success.

The scheduler sees request metadata at arrival, including deadline, criticality, model, prompt estimate, maximum token

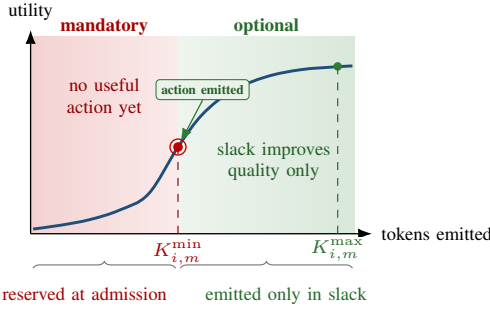


Fig. 2. Utility curve. Red dot: $K_{i,m}^{\min}$ threshold where the prefix becomes a useful action. Green dot: $K_{i,m}^{\max}$ ceiling. Tokens beyond $K_{i,m}^{\min}$ are emitted only in slack.

budget, and one or more validator-backed usefulness thresholds. It estimates token costs, output length, utility, and tool-I/O blocking from calibration profiles. The scheduler can admit, reject, defer, preempt at token boundaries, truncate optional work, or switch an arriving request to a lower-cost certified mode. For admission, $K_{i,m}^{\min}$ is a conservative reservation boundary: DATS reserves enough service to reach a validator-checkable prefix for that mode, and $K_{i,m}^{\min}$ is required to be an upper bound on the validator-accepted prefix length over the calibrated workload distribution. Evaluation success requires validator acceptance by the deadline; the scheduler reserves up to this boundary, and early validator acceptance is allowed but is not assumed by the schedulability argument. In full mode, this boundary is a structured prefix checked by the application validator. In compact mode, it is a command boundary accepted by the parser. In fallback mode, the boundary is a certified safe action (stop, no-op, or hold-state), and the bounded cost of dispatching that action to the executor is folded into the mandatory service bound used at admission.

B. Utility and Timing Semantics

Fig. 2 shows the model assumed by DATS. Utility is nondecreasing, but the scheduling objective is not total text length. Utility before $K_{i,m}^{\min}$ is insufficient for the physical loop; utility after $K_{i,m}^{\min}$ is optional and may be truncated. Deadlines are firm for mandatory usefulness: a mandatory prefix after d_i is counted as a failure for timing metrics even if the text is semantically correct. Optional tokens emitted before d_i contribute utility; optional tokens emitted after d_i are truncated in the main evaluation.

C. Resource Model

The resource model includes compute service, memory, and blocking. Service is divided into prefill chunks and decode tokens. Memory includes KV-cache state and runtime overhead associated with active requests. Blocking includes non-preemptive prefill chunks, the non-preemptive quantum imposed by the current decode batch, and tool-I/O stalls that prevent immediate dispatch. The simulator represents CPU/GPU/NPU/network fields in scenario configurations. The analysis reduces the guarantee to a single preemptive server

with bounded blocking; multi-resource behavior lies outside this guarantee.

DATS enforces memory feasibility as a reservation rather than a check of current occupancy. The KV-cache and runtime footprint $M_{i,m}(k)$ is nondecreasing in k over the calibrated profile, so for a request in mode m , let

$$\widehat{M}_{i,m}^{\min} = \max_{0 \leq k \leq K_{i,m}^{\min}} M_{i,m}(k) \quad (1)$$

be a conservative bound on the KV-cache and runtime memory needed before the validator-accepted mandatory prefix. Admission checks

$$M^{\text{unrel}}(t) + \sum_{j \in A(t)} \widehat{M}_{j,m_j}^{\min} + \widehat{M}_{i,m}^{\min} \leq M_{\max} \quad (2)$$

where $A(t)$ is the set of requests currently admitted at time t , excluding the arriving candidate i . The check sums mandatory reservations over $A(t)$, adds the candidate's reservation, and includes $M^{\text{unrel}}(t)$, the unreclaimed optional KV and runtime state from earlier admitted requests. $M^{\text{unrel}}(t)$ decreases when the allocator releases pages from truncated optional generations; reclamation latency is assumed bounded. On a new protected arrival, optional KV is truncated before admission; if the allocator cannot reclaim it immediately, its residual occupancy is charged in (2) until reclamation completes. If the projected mandatory memory demand cannot fit for the highest-quality mode, DATS tests a compact or fallback mode with lower memory demand. If no certified mode fits, the request is rejected and, for a critical request, counted as a critical failure. Optional memory growth is allowed only when doing so preserves the mandatory guard. Before optional dispatch, DATS checks that the actual memory after the next optional token plus all protected mandatory reservations remains within $M_{\max}$.

D. Online Knowledge and Failure Conditions

At arrival, the scheduler knows r_i , d_i , class L_i , prompt/model label, estimated service bounds, and optional application hints such as $K_{i,m}^{\min}$ and $K_{i,m}^{\max}$. It does not know the exact semantic content of future tokens or exact service time unless calibration bounds are correct. A critical request fails the mandatory timing objective if it is rejected or if admitted work fails to complete the selected mandatory prefix and required validation or fallback dispatch by d_i . Both admitted-critical miss and critical failure over all critical arrivals are tracked, so admission control cannot hide overload behind a zero admitted-miss number. Non-critical requests are characterized through deadline misses, utility-at-deadline, and optional-token completion.

E. Problem Statement

Inputs. A stream of requests $\{i\}$ with arrival times, deadlines, criticality L_i , candidate mode set $\mathcal{M}_i$, and per-mode service bounds, mandatory thresholds, optional ceilings, utility functions, and KV-cache profiles, together with a memory capacity $M_{\max}$ and a bounded blocking term $B_{\text{guard}}(t)$.

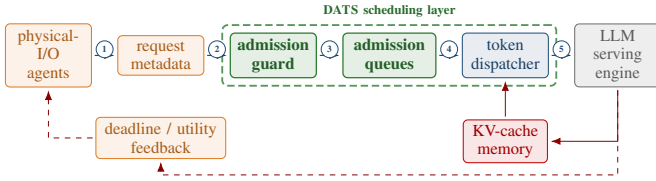


Fig. 3. Numbered arrows trace each request through metadata extraction, admission guard, queueing, dispatch, and engine emission. KV-cache occupancy (red) and deadline/utility feedback (dashed) close the loop.

Decision variables. For each request, an admit/reject decision; for admitted requests, a committed mode m_i , a KV-cache reservation, and a sequence of mandatory or optional token-quantum dispatch decisions until the request completes or its deadline elapses.

Objective. Three goals in order: protect admitted critical mandatory prefixes whenever the conservative demand and memory assumptions hold; reduce critical failure over all critical arrivals by choosing the highest-quality feasible certified mode, counting both rejections and admitted misses; and spend remaining slack on optional utility. Let A_c be critical arrivals, R_c rejected critical arrivals, and M_c admitted critical requests that miss their selected mandatory work by deadline. The critical failure ratio is

$$F_c = \frac{|R_c| + |M_c|}{|A_c|}. \quad (3)$$

Optional service is summarized by

$$O = \frac{\sum_i \max(0, k_i(d_i) - K_i^{\min})}{\sum_i \max(0, K_i^{\max} - K_i^{\min})} \quad (4)$$

over admitted requests with optional budgets, interpreted alongside admission ratio, all-arrival deadline failure, and utility-at-deadline.

Constraints. (i) The joint memory reservation in (2) holds at every t . (ii) Every admitted critical request completes mandatory prefix, validation, or fallback dispatch by d_i under the bounded service and blocking model of Section V. (iii) Optional service is scheduled only while the mandatory guard remains feasible, so optional tokens never preempt protected mandatory work.

IV. DEADLINE-AWARE TOKEN SCHEDULING

DATS is a scheduling layer between an LLM serving engine and physical-I/O agent requests. It treats prefill and each decode token as schedulable units, admits mandatory work only when the guard passes, and spends remaining slack on optional tokens with high marginal utility. In overload, DATS protects critical minimum useful output first, then truncates or rejects lower-priority work before it consumes capacity needed by physical deadlines.

Overview. DATS processes each request through six stages: (1) joint demand-and-memory admission; (2) mode selection; (3) memory reservation; (4) mandatory dispatch in EDF order; (5) optional dispatch in remaining slack; and (6) overload degradation through compact and fallback modes. Fig. 3 and Algorithm 1 make the runtime interface precise. DATS sits

Algorithm 1 DATS admission and dispatch

On arrival of request i at time t

- 1: Build candidate mode set $\mathcal{M}_i$ ordered by utility: full, compact, fallback.
- 2: **for** mode $m \in \mathcal{M}_i$ **do**
- 3: Estimate $P_{i,m}^+$, $c_{i,m}^+$ (k), $V_{i,m}^+$, $K_{i,m}^{\min}$, $K_{i,m}^{\max}$, $U_{i,m}(k)$, and $M_{i,m}(k)$.
- 4: Compute $W_{i,m}^{\min} = P_{i,m}^+ + \sum_{k=1}^{K_{i,m}^{\min}} c_{i,m}^+(k) + V_{i,m}^+$.
- 5: Set $B_{\text{guard}}(t)$ from the configured bound, or from the maximum currently active schedulable quantum if no bound is configured.
- 6: **if** mandatory-plus-blocking guard and memory reservation both pass **then**
- 7: Admit i in mode m ; reserve mandatory compute and memory.
- 8: **return**
- 9: **end if**
- 10: **end for**
- 11: Reject if no certified mode is feasible; record a critical rejection in F_c .
- 12: **Dispatch** at scheduling instant t
- 13: $C \leftarrow$ admitted, unblocked requests whose next quantum fits memory.
- 14: **if** some critical request in C has unfinished selected mandatory work **then**
- 15: Choose by EDF-compatible mandatory order.
- 16: **else if** some non-critical request in C is below its mandatory prefix **then**
- 17: Choose by EDF order subject to critical reserve.
- 18: **else**
- 19: Choose the feasible request with the best optional utility score.
- 20: **end if**
- 21: Run one bounded prefill chunk, decode token, validator step, or fallback-dispatch quantum.
- 22: Update emitted tokens, validator state, slack, utility, memory, and blocking state.
- 23: Truncate optional work whenever a future mandatory-plus-blocking guard would fail.

between the LLM serving engine and the agent's request stream: the serving engine reports bounded scheduling points, and the application reports deadline, usefulness, and optional mode metadata. These interfaces make blocking, memory, and useful-prefix demand visible to the scheduler instead of leaving them implicit in a best-effort queue.

The multi-mode extension DATS-MF makes overload handling part of admission. A critical request may carry several validator-backed modes: a full prefix, a compact schema prefix, and a safe fallback action such as stop, no-op, or hold-previous-safe-state. Each mode has its own conservative service bound, memory reservation, prefix validator, and utility. Admission chooses the highest-utility mode that preserves the protected mandatory invariant; rejection is used only when no safe mode is feasible. The selected mode defines the certified mandatory demand for that request, so the schedulability conditions of single-mode DATS apply to whichever mode is admitted.

A. Admission Control

A pure utilization check ignores deadline structure and a per-request deadline check ignores interference from other admitted protected work. DATS therefore uses a joint demand-and-memory test that captures both effects in a single guard, and the same test applies to compact and fallback modes, so mode selection at admission reduces to a sequence of guard evaluations.

For an arriving request and candidate mode m , DATS

computes the mandatory demand introduced by

$$W_{i,m}^{\min} = P_{i,m}^+ + \sum_{k=1}^{K_{i,m}^{\min}} c_{i,m}^+(k) + V_{i,m}^+, \quad (5)$$

where $P_{i,m}^+$ is a conservative prefill-service bound and $c_{i,m}^+(k)$ is a per-token upper envelope on decode-service cost. The validation-and-dispatch bound $V_{i,m}^+ = \max(V_{\text{val},i,m}^+, V_{\text{disp},i,m}^+, V_{\text{parse},i,m}^+)$ takes the maximum over the validator, fallback-dispatch, and parser execution bounds, where each term is included only for modes that exercise it: full mode uses V_{val}^+ , fallback mode uses V_{disp}^+ , and compact mode uses V_{parse}^+ . For each model/mode pair, P^+ and c^+ come from maximum held-out prefill and decode times multiplied by a fixed margin γ . We set $\gamma = 1.0$ in this work, so the guard uses the recorded bound without extra scaling. $W_i^{\min}$ and $K_{i,m}^{\min}$ are worst-case held-out bounds; if the validator accepts a shorter prefix at runtime, the request finishes early. Runs that violate the bounded-quanta assumption are excluded from the schedulability claim. The guard checks protected critical demand over the finite set of relevant future deadlines after adding $B_{\text{guard}}(t)$. During admission, the candidate's mandatory demand is included in demand, not charged again as blocking. $B_{\text{guard}}(t)$ is set to a configured constant when provided, otherwise to the maximum next schedulable prefill/decode quantum among active admitted requests. The guard logs demand, blocking, slack, outcome, and failure reason at every check. A request whose projected mandatory memory cannot fit in any certified mode is rejected before admission. Optional tokens are excluded from the schedulability claim, and non-critical requests are admitted as slack-only work. The simulator uses the reservation $\widehat{M}_{i,m}^{\min}$ and counts optional memory only when optional service is actually scheduled, so that compute-feasible requests cannot invalidate deadlines through memory pressure.

B. Dispatch Policy

Protected mandatory work must follow a strict deadline order so the admission guard remains binding, so DATS uses a mandatory mode that enforces EDF order over the protected class and an optional mode that ranks by utility within a feasibility-preserving set.

DATS uses two dispatch modes. Mandatory mode prioritizes requests with unfinished selected mandatory work. The analyzable version maintains the following invariant. Whenever a ready admitted critical request has unfinished mandatory prefix, validator, or fallback-dispatch work, the dispatcher selects the ready admitted critical mandatory job with earliest deadline. Optional tokens are scheduled only while the mandatory demand test remains feasible. Optional dispatch chooses by marginal utility once the invariant holds. This separation prevents optional long responses from consuming service needed for imminent physical deadlines.

For empirical ranking within a feasibility-preserving set, the simulator can assign a score to each candidate, where a lower

score is preferred. For mandatory work, the score combines normalized laxity, criticality, and memory pressure:

$$\text{score}_m(i, t) = w_\ell \cdot \text{norm}(d_i - t - \text{rem}_i) - w_c L_i + w_m \frac{\widehat{M}_{i,m_i}^{\min}}{M_{\max}}. \quad (6)$$

Here rem_i is the conservative remaining mandatory service for the selected mode, and $\text{norm}(\cdot)$ maps the current candidate values to $[0, 1]$ using the largest nonnegative value among feasible candidates. By the admission invariant, the laxity $d_i - t - \text{rem}_i$ is nonnegative for any protected candidate; negative values are clamped to zero for ranking only. For optional work, the score also includes marginal utility $\Delta U_i(k_i) = U_i(k_i + 1) - U_i(k_i)$:

$$\begin{aligned} \text{score}_o(i, t) = & w_s \cdot \text{norm}(d_i - t) - w_u \Delta U_i(k_i) \\ & + w_m \frac{M_{i,m_i}(k_i)}{M_{\max}}. \end{aligned} \quad (7)$$

Weights are fixed before a run. We use $w_\ell = w_s = 1.0$, $w_c = 0.75$, $w_m = 0.5$, and $w_u = 0.05$. Scores apply only within a feasibility-preserving set and never reorder protected mandatory dispatch.

C. Overload Degradation

A reject-only policy is brittle under overload: every critical arrival that cannot pass the guard in its full mode is lost, even when a smaller certified action would still meet the physical deadline. Exposing compact and safe-fallback modes as separate admission candidates converts those rejections into degraded but timely service while leaving the protected-mandatory invariant untouched.

When a new arrival would violate the mandatory guard in its full mode, DATS excludes optional suffix service from the schedulability claim and tests lower-cost certified modes before rejection. Mode selection is committed at admission: once a critical request is admitted in a given mode, DATS-MF does not revise that decision at later instants. A compact mode may use schema-constrained output with a smaller validator-derived prefix. A safe-fallback mode may bypass LLM decoding and issue a pre-certified robot stop, GUI no-op, or IoT hold-state command; it is credited as a safe action with reduced utility, not as full task completion. If a fallback is LLM-generated, it must still be accepted by the application validator before it receives timing credit. Existing mandatory reservations for admitted critical work are never reassigned to optional service. Under overload, DATS sacrifices optional detail and output quality before any admitted critical prefix can miss its deadline.

D. Complexity and Runtime Interface

With n active requests, the simulator recomputes scores in $O(n)$ per scheduling decision; admission checks over q relevant deadline intervals cost $O(nq)$. The runtime interface requires four serving-stack callbacks: estimate mandatory prefill/decode bounds, report tool/I/O or batching blockage, expose the next schedulable prefill/token quantum, and report the current KV-cache footprint. A production integration can

implement these callbacks above a paged KV-cache manager or batched decode loop.

V. SCHEDULABILITY ANALYSIS

The analysis focuses on protected critical requests on a bounded server and gives a sufficient schedulability condition for mandatory-prefix deadlines under bounded service and blocking. Multi-GPU scheduling, allocator internals, weighted optional scoring, semantic correctness, and underestimated service times fall outside this formal guarantee and are addressed empirically.

A. Assumptions

The accelerator is modeled as one preemptive server with unit service rate. Decode can be preempted at token boundaries. Prefill, decode, and batching impose bounded non-preemptive quanta represented by $B_{\text{guard}}(t)$ in the guard. The theorem covers the protected critical class only: admitted requests with $L_i = 1$ whose mandatory prefixes have not yet completed. The feasibility guarantee does not cover optional tokens, best-effort requests, or weighted utility scoring; these are slack-only policies.

Guard checks occur at critical-request admission, before optional dispatch, after prefill/decode or unblock events, and before truncation or mode-selection events spend service. A new arrival checks before entering the protected set. Between such events, any prefill, batching, tool, or I/O delay that can prevent dispatch must be represented as bounded service or blocking.

At admission and dispatch instant t , the scheduler maintains the online admitted set $A(t)$. For an admitted request i , let m_i be its committed mode and write $W_i^{\min} \equiv W_{i,m_i}^{\min}$. Let $x_i(t)$ be completed mandatory service, including partial prefill or decode work, and let

$$R_i(t) = \max(0, W_i^{\min} - x_i(t)) \quad (8)$$

be the remaining mandatory work. The demand calculation treats service as fungible across prefill, decode, and validator phases within $W_i^{\min}$; the per-phase sequencing is preserved at dispatch but does not appear in $R_i(t)$. Validator execution runs after the prefix is emitted, and fallback dispatch runs after the fallback decision; the proof needs only that the total mandatory service $W_i^{\min}$ is bounded and that EDF order is preserved across requests, not across phases of one request. Work released before t is counted only through its remaining demand $R_i(t)$; service already executed is not charged a second time. Work that extends past d_i is not permitted for a protected prefix. The admission invariant is that, after every admission, mode-selection decision, or rejection, each protected critical request has fixed certified mandatory demand, feasible mandatory memory reservation, and deadline. Critical arrivals that do not satisfy this invariant are recorded as critical rejections or failures and are not added to the protected set. Optional service runs only when the invariant remains true.

B. Mandatory Demand

Let $C_p(t)$ be the protected critical subset of $A(t)$ whose mandatory prefix is incomplete after the current admission or mode-selection decision. For any future deadline $b > t$, define remaining protected mandatory demand by

$$W_t(b) = \sum_{i \in C_p(t): d_i \leq b} R_i(t). \quad (9)$$

When a critical candidate is being tested, $C_p(t)$ includes the candidate only for the tentative admitted set, so the candidate's mandatory prefix is charged in $W_t(b)$. The candidate is not also counted in $B_t(b)$ during admission. The reason is that the candidate's mandatory work is fully captured by its demand contribution to $W_t(b)$; its non-preemptive quantum enters $B_t(\cdot)$ only after admission, for later guard checks against other arriving requests. The relevant deadline set is the finite set of protected critical mandatory deadlines greater than t , plus the candidate deadline during an admission check. The implementation checks this finite set rather than all hyperperiod intervals, since active admitted jobs are already released at time t . Let $B_t(b)$ be the implemented guard blocking term. In the simulator this is a scalar $B_{\text{guard}}(t)$: either a configured constant or the maximum next schedulable prefill/decode quantum among active admitted requests. Self-blocking from tool or I/O calls within a request is outside the theorem and must be folded into $W_i^{\min}$. Integration with tool-using agents requires either this folding or an extension of the theorem.

Theorem 1 (mandatory-mode sufficiency). For a finite online admitted set $A(t)$ on a unit-rate server with token-boundary preemption, conservative service bounds, feasible mandatory memory, and the stated admission invariant, if every relevant future deadline $b > t$ in the protected set satisfies

$$W_t(b) + B_t(b) \leq b - t, \quad (10)$$

then token-level EDF, or the EDF-compatible mandatory mode in DATS, completes each protected critical request's selected mandatory prefix and bounded validator/action-dispatch work by its deadline.

Corollary 1 (online admission). If the guard $W_t(b) + B_t(b) \leq b - t$ holds at every admission instant for the updated protected set and no already-admitted critical request misses prior to t , then by induction over the sequence of admission events no protected critical mandatory deadline is missed.

Proof. Assume the earliest missed protected critical mandatory deadline d_j . Let t be the last guard event before that miss. At t , protected carry-in work due by d_j contributes only remaining mandatory demand $R_i(t)$, and newly admitted protected work due by d_j has been included at admission. Each unit of time in $[t, d_j]$ is therefore either spent serving protected mandatory work with deadline no later than d_j or counted against the guard blocking term $B_t(d_j)$, since the model bounds every other source of delay by $B_t(d_j)$. EDF dispatches protected mandatory work in increasing-deadline order, so no service is spent on protected work with deadline strictly

greater than d_j while protected work due by d_j remains ready. Therefore, a miss at d_j would imply $W_t(d_j) + B_t(d_j) > d_j - t$, contradicting the maintained guard.

C. Token-Level Blocking

Proposition 1 (dispatch blocking). Suppose decode preemption is available at token boundaries, each token service time is bounded by $q_{\max}$, prefill chunks are bounded by B_p , and batching imposes a bounded non-preemptive quantum B_b . Case (a): if prefill, decode, and batching impose mutually exclusive non-preemptive segments, so that at most one of $q_{\max}$, B_p , B_b is in effect when a critical mandatory request becomes ready, then the request can be delayed by at most $\max(q_{\max}, B_p, B_b)$ before the EDF-compatible dispatcher can select it. Case (b): if these segments can serialize at the dispatch instant, a newly ready critical request can wait through at most one of each type before the dispatcher selects it; the worst case is $B_p + q_{\max} + B_b$. Both cases exclude self-blocking caused by the request’s own tool or I/O calls. Under request-level non-preemptive decode, the blocking term can be as large as the remaining decode budget of the currently running request.

D. Critical Isolation

Proposition 2 (slack-only optional isolation). If DATS schedules optional tokens only when the demand condition remains satisfied after reserving all critical mandatory work, then optional best-effort requests cannot be the direct cause of a critical mandatory deadline miss under the same service-bound assumptions. Any critical miss must come from violated assumptions, such as underestimated service time, unmodeled blocking, memory eviction cost, or runtime failure, rather than from planned optional service.

VI. IMPLEMENTATION

The primary evaluator is a deterministic discrete-event simulator that tests admission, mandatory/optional dispatch, memory guards, and overload accounting under controlled workloads. Scenario configuration produces task graphs, and the simulator records request outcomes and guard events used to compute the reported tables and figures. Local model probes and native llama.cpp/Vulkan runs [13], [14] provide server-timing fields and client-side bounded-generation dispatch alongside the simulator.

A. Simulator Structure

Each run constructs requests from a scenario configuration and seed. A request carries arrival time, deadline, criticality, prefill cost, token-cost profile, $K_i^{\min}$, $K_i^{\max}$, utility, memory, optional blocking, and weakly-hard metadata when applicable. The event loop advances to arrivals, unblocks, and scheduling completions; event rows record admission, rejection, optional truncation, and guard checks. The implementation contains request-level and token-level baselines; guarded variants receive the same admission guard as DATS so that the comparison separates dispatch policy from the

presence of admission control. DATS variants include the main implementation (EDF-compatible ordering for protected critical mandatory work, weighted scores only for feasibility-preserving optional or non-critical choices) and a pure DATS-EDF variant. Table III lists the nominal workload ranges. The seed fixes arrivals and per-request profiles. Two multipliers scale overload: a load multiplier $\in \{0.75, 1.0, 1.25, 1.5\}$ on arrival rate, and a service-scale multiplier $\in \{0.5, 1.0, 2.0\}$ on prefill and decode cost.

TABLE III
NOMINAL WORKLOAD PARAMETERS. RANGES ARE PER-REQUEST VALUES OVER TEN SEEDS; MOBILE-GUI IS USED AS A NON-CRITICAL UTILITY WORKLOAD. PREFILL RANGES FROM 3 TO 30 MS, DECODE SERVICE FROM 4 TO 35 MS, AND MEMORY STRESS PERTURBS SCENARIO KV/CACHE PROFILES.

Scenario	Req./win.	Crit.	Deadline	$K^{\min}$	$K^{\max}$
Robot	48/5 s	1.00	50 to 500 ms	2 to 8	8 to 28
Mobile-GUI	44/12 s	0.00	1 to 5 s	6 to 20	20 to 80
IoT sensor	60/10 s	1.00	0.1 to 2 s	2 to 12	10 to 50
Mixed crit.	64/10 s	0.25	0.11 to 4 s	2 to 18	10 to 70

B. Measurement Controls

The simulator is seed controlled and records request outcomes, guard events, and summary statistics. Calibration sanity checks verify deterministic replay, request-accounting identities, ratio bounds, service-scale, hardware-measurement, and overhead-model checks. Compute service, memory occupancy, and blocking are modeled as bounded quantities; integration with paged-KV allocators, prompt-cache hits, semantic action correctness, and robot middleware is handled by the calibration and runtime measurements rather than the simulator.

VII. EVALUATION

The evaluation tests three claims: the admission and dispatch guard protects admitted critical mandatory prefixes; overload remains visible through rejection, optional-token loss, and all-arrival deadline metrics; and the mechanism is stable under load, service scaling, jitter, noise, and synthetic overhead. The controlled workloads model robot control decisions, mobile-GUI actions, IoT sensor scheduling, and mixed-criticality requests, with mobile-GUI included as a non-critical utility workload. Baselines include request-level and token-level EDF, LLF, round-robin, SPOF, deadline-oblivious batching, and a segmented TimelyLLM-style approximation; guarded variants receive the same critical admission guard and optional-past-deadline truncation rule as DATS. Reported metrics include admitted-critical miss, critical failure over all critical arrivals, deadline miss, weakly-hard violation, optional-token completion, utility at deadline, admission ratio, latency, memory, and simulated scheduler overhead.

Recorded simulator outcomes are the source of the main scheduling results. Calibration is reported separately: service scaling, CPU Transformers measured-timing replay [42], a CPU dispatch probe, semantic $K_i^{\min}$ probes, CPU RSS/KV rows, closest-prior comparisons, and a native llama.cpp/Vulkan server-timing run. The scheduler-overhead figure is an operation-count model.

Workload generation is synthetic and seed controlled. Within each scenario, the seed fixes arrivals, service times, prefix lengths, optional budgets, and memory profiles drawn from the Table III ranges; workload multipliers scale load. Physical-I/O timing uses periodic robot and sensor streams, bursty GUI actions, and mixed-criticality streams.

Metric definitions are consistent across reported tables. Utility-at-deadline is zero unless the request completes its selected mandatory work by its physical deadline; rejected arrivals receive zero reported utility, and admitted-only utility is exported separately. Critical failure counts rejected critical arrivals plus admitted critical requests that miss their selected mandatory work; scenarios with no generated critical arrivals are N/A. Mandatory success requires timely prefix emission plus validator acceptance or fallback-dispatch completion for the selected mode. Weakly-hard ratios count generated request streams and treat rejections as misses; admitted-only values appear as diagnostic columns.

A. Baselines and Fairness

The baselines separate scheduling granularity, admission control, and mandatory/optional awareness. Request-level FCFS and EDF use whole-request queues; token-level EDF and LLF give decode-level priority but do not reserve mandatory prefixes. Round-robin and SPOF represent fairness-oriented and size-oriented alternatives. Deadline-oblivious batching is throughput-first, and the segmented baseline follows the published TimelyLLM design principles. Guarded EDF, guarded LLF, and guarded segmented receive the same admission and truncation guard as DATS.

The workload-tuned guarded fallback baseline is the strongest single comparison for DATS-MF. It uses the same admission guard but adds workload-specific fallback thresholds tuned per trace. It is a strong fallback-only baseline rather than an analyzable scheduler, and does not provide per-mode validator interfaces, mode-specific memory reservations, or an EDF-compatible protected mode.

All policies run on identical arrivals, service profiles, utility curves, memory profiles, and seeds, and reject only through the configured guard path. Rejected critical arrivals are counted in critical failure because a scheduler can otherwise achieve zero admitted misses by admitting too few requests.

B. Workloads and Reproducibility

The workloads are deterministic, parameterized task sets that exercise scheduling behavior under controlled timing models. Semantic prefix probes are reported separately to validate the output formats used by those timing workloads. A llama.cpp validator run [13] covers 150 robot, mobile-GUI, and IoT-sensor tasks and 16,021 prefix rows. At a 24-token scheduled prefix, free-form prompts have 0% validator acceptance in all domains. Fallback modes are validator-accepted and safe in all domains; structured modes reach task-valid acceptance of 76% for mobile-GUI, 50% for robot, and 24% for IoT; schema-constrained modes reach 48% for IoT

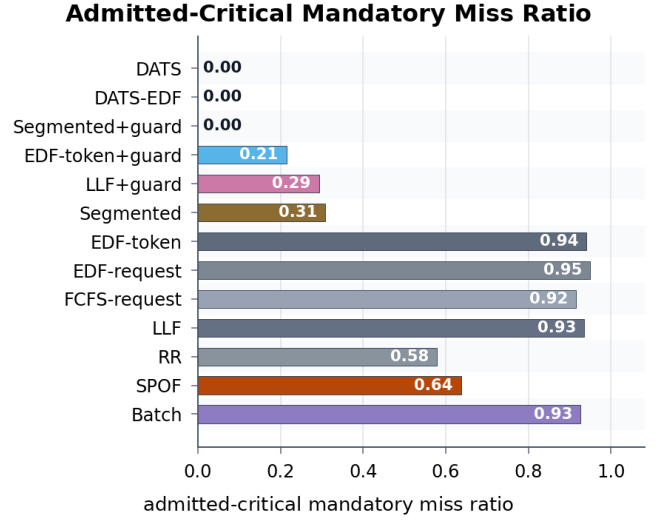


Fig. 4. Critical mandatory miss ratio across nominal workloads.

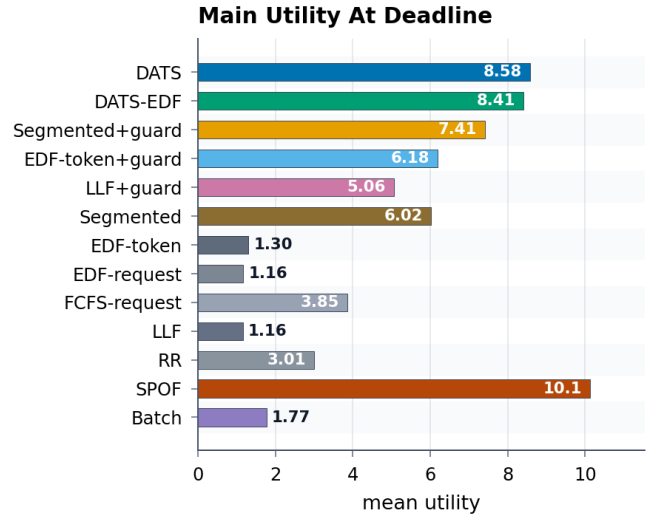


Fig. 5. Mean utility at deadline. Utility is zero before $K_i^{\min}$, and rejected arrivals receive zero utility.

and 26% for robot. These measurements motivate validator-derived $K_i^{\min}$ for constrained formats rather than a raw token count as a semantic target.

C. Nominal Results

Fig. 4 to Fig. 7 and Table IV show the nominal result after adding demand-bound admission, mandatory memory reservation, optional-token truncation, and admission-aware baselines. DATS reports zero admitted-critical misses in all nominal scenarios with critical requests; mobile-GUI has no generated critical requests and is N/A for critical metrics. Its macro-average critical-failure ratio is 0.118 over scenarios with critical requests. DATS-EDF is 0.122 and guarded segmented is 0.120, with a paired 95% CI on the difference of $[-0.0041, 0.0013]$. Guarded token-level EDF and guarded

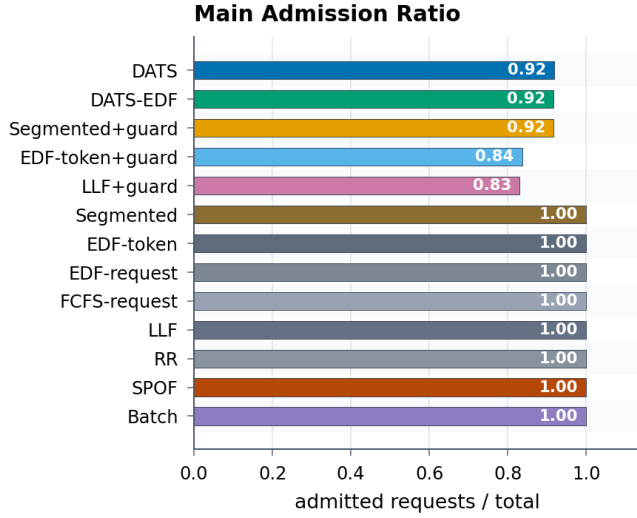


Fig. 6. Nominal admission ratio. Guarded policies reject some arrivals under overload; the robot trace has the highest admission-control cost.

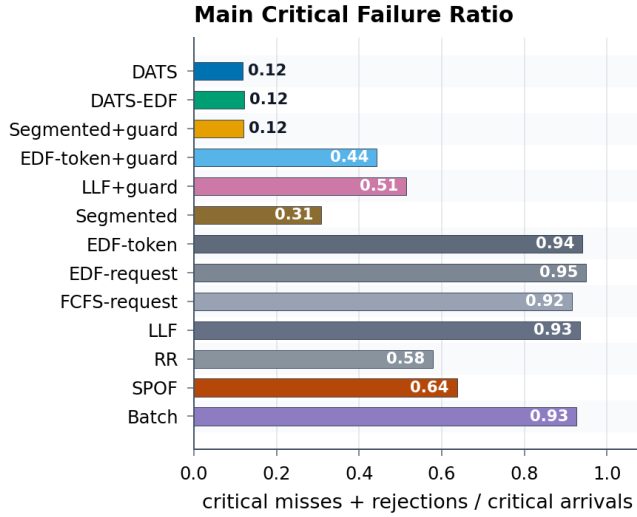


Fig. 7. Critical failure ratio over all critical arrivals (rejection plus admitted-critical miss). No-critical scenarios appear as N/A in tables.

LLF, which do not preserve a mandatory-prefix guarantee, are much worse at 0.443 and 0.514.

Guarded segmented receives the same admission guard and truncation rule, so the closeness confirms that the guard, not the dispatch policy, protects admitted prefixes. DATS reports mandatory prefix demand, memory reservation, rejection accounting, slack-only optional service, and EDF-compatible protected mode at admission. Guarded segmented combines these properties only by composition. In all reported DATS rows, protected critical mandatory tokens are dispatched in EDF-compatible order, and weighted scores apply only to feasibility-preserving non-critical or optional choices. Table VI reports paired intervals for the closest nominal, multi-mode, and vLLM stress comparisons.

These results also show the cost of protection. Fig. 5 reports

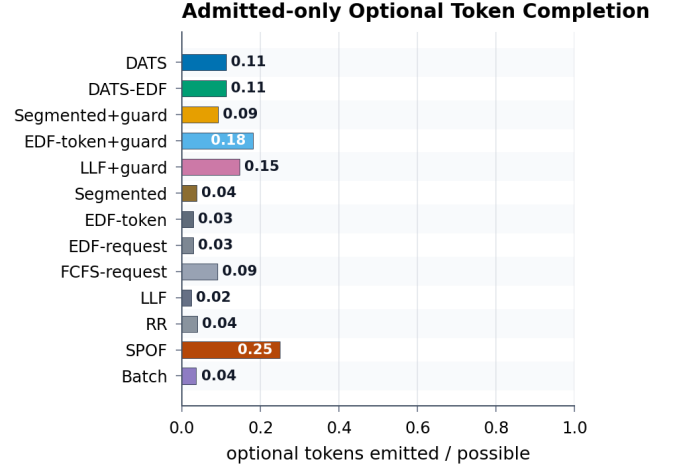


Fig. 8. Admitted-only deadline-limited optional token completion. Late optional tokens are not credited.

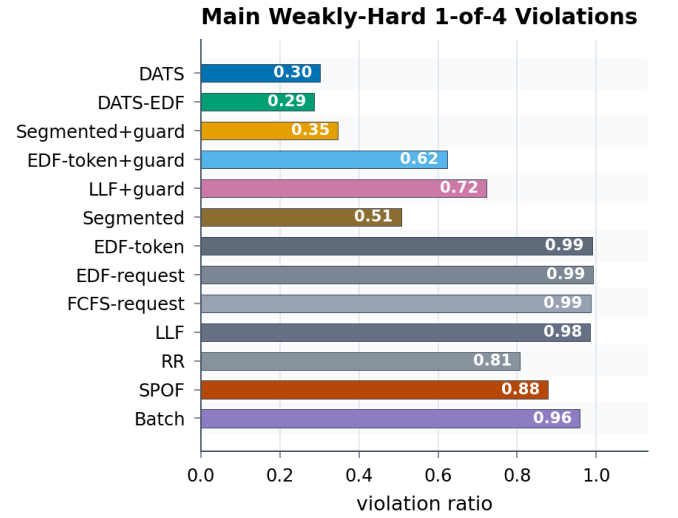


Fig. 9. Weakly-hard (1, 4) violation ratio over request streams. DATS reduces aggregate violations relative to EDF and SPOF while still exposing residual non-critical misses.

utility at the deadline, Fig. 6 reports admission loss, and Fig. 8 shows that DATS emits 11.2% of possible optional tokens before deadline on average over all nominal seed-controlled workloads. The robot workload rejects 25.4% of critical arrivals. Mixed-criticality and mobile-GUI workloads have overall deadline-failure ratios of 0.408 and 0.277. DATS lowers average all-arrival deadline failure to 0.251, compared with 0.297 for guarded segmented, 0.564 for guarded token-level EDF, and 0.949 for unguarded token-level EDF.

Fig. 9 reports the request-stream weakly-hard violations. The all-arrival request-stream weakly-hard (1, 4) violation ratio is 0.301 for DATS, 0.347 for guarded segmented, 0.623 for guarded token-level EDF, and 0.990 for unguarded token-level EDF. Over scenarios with critical requests, the corresponding critical request-stream weakly-hard (1, 4) ratios are 0.096, 0.099, 0.504, and 0.993.

TABLE IV

NOMINAL SIMULATION RESULTS. MEANS OVER TEN SEEDS; CRITICAL AVERAGES EXCLUDE MOBILE-GUI. D=DATS; G=GUARDED SEGMENTED.

Scenario	D crit. fail	D deadl.	D opt. comp.	G crit. fail	G deadl.
IoT sensor	0.063	0.063	0.176	0.063	0.063
Mixed criticality	0.038	0.408	0.027	0.038	0.581
Mobile GUI	N/A	0.277	0.030	N/A	0.286
Robot	0.254	0.254	0.215	0.258	0.258
Crit.-request avg.	0.118	0.242	0.139	0.120	0.301

TABLE V

DATS-MF MODE MIX OVER ALL GENERATED REQUESTS. MOBILE-GUI HAS NO CRITICAL ARRIVALS, SO ITS REJECTIONS AFFECT UTILITY AND DEADLINE METRICS ONLY. UTILITY IS MEAN AT DEADLINE.

Workload	Full	Compact	Fallback	Rejected	Utility
IoT sensor	0.632	0.112	0.161	0.096	6.02
Mixed crit.	0.497	0.022	0.046	0.436	3.29
Mobile-GUI	0.474	0.000	0.000	0.526	3.59
Robot	0.467	0.232	0.246	0.055	3.62

The multi-mode design addresses the main residual failure mode in the reject-only policy: critical arrivals that are infeasible in full mode. In deterministic sweeps over 155,500 requests, DATS-MF reduces critical failure over all critical arrivals from 0.402 for full-only DATS and 0.399 for guarded segmentation without fallback to 0.119, by using compact or fallback modes. A workload-tuned guarded fallback baseline reaches 0.102 critical failure and 0.368 all-arrival failure. DATS-MF trails that baseline by 0.017 on critical failure but requires no per-workload tuning and provides per-mode validation, mode-specific memory reservation, and an EDF-compatible protected class. Table V shows how often the policy uses full, compact, fallback, and rejection outcomes.

D. Ablation

Ablation results show which mechanisms matter most in the nominal simulator workloads. Removing mandatory protection increases the average admitted-critical miss ratio from 0.000 to 0.684 across the ablation workloads with critical requests, while request-stream critical weakly-hard (1,4) violations rise from 0.089 to 0.898. Removing the criticality term increases the average critical-failure ratio from 0.115 to 0.278, mainly through mixed-criticality conflicts. Removing memory pressure or utility scoring does not change admitted-critical misses in this table. Mandatory-prefix admission is the dominant mechanism; memory and utility terms shape only the overload region in the stress and ablation settings.

E. Stress, Robustness, and Sensitivity

Load sensitivity, stress, and robustness workloads support the same bounded claim. DATS maintains zero admitted-critical misses in generated stress tests and under arrival jitter, service noise, and combined noise; counting rejected critical requests in the failure metric reports the remaining overload cost. The sensitivity sweeps show where the assumptions become tight: macro-average DATS critical failure rises from 0.057 to 0.280 across the $K_i^{\min}$ sweep and from 0.117 to 0.525 as the decode quantum grows from one to four tokens.

TABLE VI

CLOSEST PAIRED CRITICAL-FAILURE COMPARISONS. Δ = OURS - BASE; NEGATIVE Δ FAVORS OUR POLICY. WORKLOAD-TUNED GUARDED FALLBACK TUNES A SINGLE FALLBACK THRESHOLD PER WORKLOAD.

Comparison	Ours	Base	Δ [95% CI]
DATS vs. guarded segmented	0.118	0.120	-0.001 [-0.004, 0.001]
DATS-MF vs. full-only DATS	0.119	0.402	-0.284 [-0.306, -0.261]
DATS-MF vs. workload-tuned fallback	0.119	0.102	+0.017 [0.006, 0.027]
vLLM DATS-MF vs. full-only	0.238	0.925	-0.688 [-0.869, -0.506]
vLLM DATS-MF vs. workload-tuned fallback	0.238	0.000	+0.238 [0.042, 0.433]

Infeasible request streams still require rejection or a lower-cost certified mode.

F. Runtime and Calibration Evidence

Runtime experiments separate calibration from scheduling claims. Theorem 1 covers a single preemptive server with bounded blocking and feasible memory; the runtime measurements below test behavior beyond that model. CPU measured-timing replay gives DATS critical failure 0.155, compared with 0.161 for guarded segmented and 0.921 for FCFS-request. Native llama.cpp/Vulkan [13], [14] confirms timing, slot, token, and memory telemetry under unstressed and stressed settings. The in-process engine loop gives DATS-MF 0.317 critical failure compared with 0.631 for full-only DATS, exercising mode selection and reclamation without a serving engine in the loop.

The vLLM/ROCm stress test uses vLLM 0.21.0+ROCm [2], [15], [16] with facebook/opt-125m [43] on an AMD RX 7800 XT and processes 5,760 requests and 13,626 token events through the scheduler-hook API. In overload, DATS-MF has 0.238 critical failure, compared with 0.925 for full-only DATS, guarded segmentation without fallback, and FCFS. Workload-tuned guarded fallback reaches 0.0 critical failure but requires per-trace threshold tuning; DATS-MF uses the same admission, reservation, and validation structure across all workloads.

G. Discussion

DATS protects admitted critical mandatory prefixes under bounded service, blocking, and memory reservation; DATS-MF extends this through compact and certified fallback modes. The analysis covers the EDF-compatible mode; weighted scoring, multi-resource behavior, batching, allocator effects, and the choice of $K_i^{\min}$ from a parser, schema, validator, or fallback boundary are evaluated empirically. The conservative bounds $\gamma = 1.0$, $K_{i,m}^{\min}$, and the maximum-quantum $B_{\text{guard}}(t)$ trade peak utilization for analytical safety. Workloads with self-blocking tool or I/O calls remain outside the theorem's scope and would require folding the bounded tool cost into the mandatory service bound.

VIII. CONCLUSION

DATS gives LLM physical-I/O agents a real-time scheduling layer that reserves validator-backed mandatory prefixes and spends optional tokens only as slack. DATS-MF adds

compact and certified fallback modes with per-mode validation and mode-specific memory reservation, so a critical request can degrade gracefully before rejection. We prove a sufficient demand-bound condition for the protected mandatory mode under bounded service, blocking, and memory. Runtime measurements with llama.cpp/Vulkan and vLLM/ROCm confirm DATS-MF on a real serving stack. DATS-MF lowers critical failure from 0.402 to 0.119 on deterministic sweeps and from 0.925 to 0.238 under vLLM/ROCm stress.

ACKNOWLEDGMENT

The work was supported by the US National Science Foundation through grants CNS-2601685 and CNS-2602744, and by the US Office of Naval Research through grant N00014-23-1-2151.

REFERENCES

- [1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-Based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/you>
- [2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [3] N. Ling, G. Chen, and L. Zhong, "TimelyLLM: Segmented LLM serving system for time-sensitive robotic applications," 2024. [Online]. Available: <https://arxiv.org/abs/2412.18695>
- [4] N. Ling, G. Chen, A. Khandelwal, and L. Zhong, "TimelyLLM: Time-sensitive LLM serving system for physical-I/O limited agents," in *Proceedings of the 24th Annual International Conference on Mobile Systems, Applications and Services (MobiSys 2026)*. ACM, 2026, pp. 261–278. [Online]. Available: <https://doi.org/10.1145/3745756.3809203>
- [5] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973. [Online]. Available: <https://doi.org/10.1145/321738.321743>
- [6] M. L. Dertouzos, "Control robotics: The procedural control of physical processes," in *Proceedings of the IFIP Congress*, 1974, pp. 807–813.
- [7] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011. [Online]. Available: <https://doi.org/10.1007/978-1-4614-0676-1>
- [8] W.-K. Shih, J. W. S. Liu, and J. Y. Chung, "Algorithms for scheduling imprecise computations with timing constraints," *SIAM Journal on Computing*, vol. 20, no. 3, pp. 537–552, 1991. [Online]. Available: <https://doi.org/10.1137/0220035>
- [9] S. Liu, S. Yao, X. Fu, R. Tabish, S. Yu, A. Bansal, H. Yun, L. Sha, and T. F. Abdelzaher, "On removing algorithmic priority inversion from mission-critical machine inference pipelines," in *2020 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2020, pp. 319–332. [Online]. Available: <https://doi.org/10.1109/RTSS49844.2020.00037>
- [10] W. Kang, K. Lee, J. Lee, I. Shin, and H. S. Chwa, "LaLaRAND: Flexible layer-by-layer CPU/GPU scheduling for real-time DNN tasks," in *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2021, pp. 329–341. [Online]. Available: <https://doi.org/10.1109/RTSS52674.2021.00038>
- [11] M. Ji, S. Yi, C. Koo, S. Ahn, D. Seo, N. D. Dutt, and J.-C. Kim, "Demand layering for real-time DNN inference with minimized memory usage," in *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2022, pp. 291–304. [Online]. Available: <https://doi.org/10.1109/RTSS55097.2022.00033>
- [12] W. Kang, J. Lee, Y. Lee, S. Oh, K. Lee, and H. S. Chwa, "RT-Swap: Addressing GPU memory bottlenecks for real-time multi-DNN inference," in *2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2024, pp. 373–385. [Online]. Available: <https://doi.org/10.1109/RTAS61025.2024.00037>
- [13] G. Gerganov and contributors, "llama.cpp: LLM inference in C/C++," <https://github.com/ggml-org/llama.cpp>, 2023.
- [14] Khronos Group, "Vulkan specification," <https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html>, 2026.
- [15] vLLM Project, "vLLM engine arguments: Custom scheduler class," https://docs.vllm.ai/en/stable/configuration/engine_args/, 2026.
- [16] Advanced Micro Devices, "ROCm documentation," <https://rocm.docs.amd.com/>, 2026.
- [17] Y. Li, Z. Li, W. Yang, and C. Liu, "RT-LM: Uncertainty-aware resource management for real-time inference of language models," in *2023 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2023, pp. 158–171. [Online]. Available: <https://doi.org/10.1109/RTSS59052.2023.00023>
- [18] Y. Akhauri, A. F. AbouElhamayed, Y. Gao, C.-C. Chang, N. Jain, and M. S. Abdelfattah, "TokenButler: Token importance is predictable," 2025. [Online]. Available: <https://arxiv.org/abs/2503.07518v1>
- [19] H. Baek, J. Kwak, and J. Lee, "Non-preemptive real-time multiprocessor scheduling beyond work-conserving," in *2020 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2020, pp. 102–114. [Online]. Available: <https://doi.org/10.1109/RTSS49844.2020.00020>
- [20] H. Shi and L. Chen, "On batching task scheduling," in *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2022, pp. 79–91. [Online]. Available: <https://doi.org/10.1109/RTSS55097.2022.00017>
- [21] T. Zhao, W. Li, and A. Y. Zomaya, "Real-time scheduling with predictions," in *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2022, pp. 331–343. [Online]. Available: <https://doi.org/10.1109/RTSS55097.2022.00036>
- [22] N. Vreman, R. Pates, and M. Maggio, "WeaklyHard.jl: Scalable analysis of weakly-hard constraints," in *2022 IEEE 28th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2022, pp. 228–240. [Online]. Available: <https://doi.org/10.1109/RTAS54340.2022.00026>
- [23] G. von der Brüggen, N. Piatkowski, K.-H. Chen, J.-J. Chen, K. Morik, and B. B. Brandenburg, "Efficiently approximating the worst-case deadline failure probability under EDF," in *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2021, pp. 214–226. [Online]. Available: <https://doi.org/10.1109/RTSS52674.2021.00029>
- [24] L. Niu and D. B. Rawat, "Energy-constrained standby-sparing for weakly hard real-time systems," in *2020 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2020, pp. 257–269. [Online]. Available: <https://doi.org/10.1109/RTSS49844.2020.00032>
- [25] S. Liu, X. Fu, M. B. Wigness, P. David, S. Yao, L. Sha, and T. F. Abdelzaher, "Self-cueing real-time attention scheduling in criticality-aware visual machine perception," in *2022 IEEE 28th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2022, pp. 173–186. [Online]. Available: <https://doi.org/10.1109/RTAS54340.2022.00022>
- [26] Y. Hu, I. Gokarn, S. Liu, A. Misra, and T. F. Abdelzaher, "Algorithms for canvas-based attention scheduling with resizing," in *2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2024, pp. 348–359. [Online]. Available: <https://doi.org/10.1109/RTAS61025.2024.00035>
- [27] W. Kang, S. Chung, J. Y. Kim, Y. Lee, K. Lee, J. Lee, K. G. Shin, and H. S. Chwa, "DNN-SAM: Split-and-merge DNN execution for real-time object detection," in *2022 IEEE 28th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2022, pp. 160–172. [Online]. Available: <https://doi.org/10.1109/RTAS54340.2022.00021>
- [28] D. Kang, S. Lee, H. S. Chwa, S.-H. Bae, C. M. Kang, J. Lee, and H. Baek, "RT-MOT: Confidence-aware real-time scheduling framework for multi-object tracking tasks," in *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2022, pp. 318–330. [Online]. Available: <https://doi.org/10.1109/RTSS55097.2022.00035>
- [29] J. Bakita and J. H. Anderson, "Hardware compute partitioning on NVIDIA GPUs," in *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2023, pp. 54–66. [Online]. Available: <https://doi.org/10.1109/RTAS58335.2023.00012>
- [30] —, "Enabling GPU memory oversubscription via transparent paging to an NVMe SSD," in *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2022, pp. 370–382. [Online]. Available: <https://doi.org/10.1109/RTSS55097.2022.00039>
- [31] Y. Ni, T. Ma, J. Chen, C. Yang, S. Ye, Y. Xu, Y. Jin, and A. Zou, "HARD: Hardening real-time scheduling and analysis for accelerator enabled computing," in *2025 IEEE 31st Real-Time and Embedded Technology*

- and Applications Symposium (RTAS). IEEE, 2025, pp. 389–401. [Online]. Available: <https://doi.org/10.1109/RTAS65571.2025.00043>
- [32] Y. Tang, Z. Feng, N. Guan, X. Jiang, M. Lv, Q. Deng, and W. Yi, “Response time analysis and priority assignment of processing chains on ROS2 executors,” in *2020 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2020, pp. 231–243. [Online]. Available: <https://doi.org/10.1109/RTSS49844.2020.00030>
- [33] T. Blaß, D. Casini, S. Bozhko, and B. B. Brandenburg, “A ROS 2 response-time analysis exploiting starvation freedom and execution-time variance,” in *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2021, pp. 41–53. [Online]. Available: <https://doi.org/10.1109/RTSS52674.2021.00016>
- [34] H. Teper, M. Günzel, N. Ueter, G. von der Brüggen, and J.-J. Chen, “End-to-end timing analysis in ROS2,” in *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2022, pp. 53–65. [Online]. Available: <https://doi.org/10.1109/RTSS55097.2022.00015>
- [35] R. Li, T. Hu, X. Jiang, L. Li, W. Xing, Q. Deng, and N. Guan, “ROSGM: A real-time GPU management framework with plug-in policies for ROS 2,” in *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2023, pp. 93–105. [Online]. Available: <https://doi.org/10.1109/RTAS58335.2023.00015>
- [36] D. Enright, Y. Xiang, H. Choi, and H. Kim, “PAAM: A framework for coordinated and priority-driven accelerator management in ROS 2,” in *2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2024, pp. 81–94. [Online]. Available: <https://doi.org/10.1109/RTAS61025.2024.00015>
- [37] H. Teper, O. Bell, M. Günzel, C. Gill, and J.-J. Chen, “Reconciling ROS 2 with classical real-time scheduling of periodic tasks,” in *2025 IEEE 31st Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2025, pp. 177–189. [Online]. Available: <https://doi.org/10.1109/RTAS65571.2025.00027>
- [38] K. M. Wilson, A. A. Arafat, J. Baugh, R. Yu, and Z. Guo, “Physics-informed mixed-criticality scheduling for F1Tenth cars with preemptable ROS 2 executors,” in *2025 IEEE 31st Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2025, pp. 215–227. [Online]. Available: <https://doi.org/10.1109/RTAS65571.2025.00030>
- [39] D. Roy, S. Ghosh, Q. Zhu, M. Caccamo, and S. Chakraborty, “GoodSpread: Criticality-aware static scheduling of CPS with multi-QoS resources,” in *2020 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2020, pp. 178–190. [Online]. Available: <https://doi.org/10.1109/RTSS49844.2020.00026>
- [40] P. Wu, C. Fu, T. Wang, M. Li, Y. Zhao, C. J. Xue, and S. Han, “Composite resource scheduling for networked control systems,” in *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2021, pp. 162–175. [Online]. Available: <https://doi.org/10.1109/RTSS52674.2021.00025>
- [41] T. Braun and S. Altmeyer, “Handling system overloads: An empirical evaluation of deadline-miss handling strategies,” in *2025 IEEE 31st Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2025, pp. 228–240. [Online]. Available: <https://doi.org/10.1109/RTAS65571.2025.00031>
- [42] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2020, pp. 38–45. [Online]. Available: <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- [43] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, “OPT: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.